UCS503P - Software Engineering Project

Civic Issue Reporter

A Role-Based Web Platform for Reporting, Routing, and Resolving
Campus Infrastructure Issues at TIET

Submitted by:

1024030227 Lavanya Sharma
1024030309 Parismita Thapa
1024030389 Rudra Yadav
1024030291 Nadeem Esrar

Group: 3C22 BE Third Year, Computer Engineering (COE)

Submitted to:

Ms. Anushka

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1

Introduction

1.1 Background and Context

Campus infrastructure faults at the Thapar Institute of Engineering and Technology (TIET) – broken equipment, plumbing leaks, faulty wiring, damaged furniture, non-functional washrooms, and unsafe walkways – are today reported informally: through hostel WhatsApp groups, verbal complaints to wardens, or ad-hoc emails to department staff. TIET's campus spans multiple hostels, academic blocks, and shared facilities (the library, cafeterias, the sports complex) maintained by different departments. Because the reporting channel is informal and department-specific, the effective bottleneck in resolving these issues is not maintenance capacity itself but *issue visibility and routing*: nobody has a shared, authoritative view of what has already been reported, to whom, and at what stage it stands.

The Civic Issue Reporter project applies standard software engineering practice [1] – requirement analysis, system design, and structured, iterative development – to this real campus problem. The broader motivation is to improve campus infrastructure governance by reducing the time and effort required to resolve civic issues reported by students and faculty. The immediate engineering objective is to design and implement a measurable, deployable web platform for efficient, department-routed issue resolution across academic blocks, hostels, the library, cafeterias, the sports complex, and other common areas.

1.1.1 Problem Statement

The absence of a structured reporting channel produces several concrete problems:

- **No single source of truth:** the same issue is reported independently to multiple people, with no shared record of what has already been flagged or fixed.
- **No accountability or tracking:** a reporter has no way to check whether a complaint was received, let alone what stage it is at.
- **No severity-aware prioritization:** a loose wire in a washroom and a broken chair in a common room compete for the same informal attention, with no structural way to surface safety-critical issues first.
- **Redundant effort:** without a shared, searchable list of open issues, students and faculty re-report problems that are already known, and administrators re-investigate what has already been logged.
- **No institutional visibility:** there is no aggregate view – by building, department, or

category – that lets facilities management identify recurring or high-impact problem areas.

1.2 Project Objectives

The primary goal of the project is to design and deploy a role-based web platform that gives the TIET campus community a structured, trackable channel for reporting and resolving infrastructure issues, while reducing duplicate reporting and improving response time to high-priority faults. This goal is decomposed into the following SMART (Specific, Measurable, Achievable, Relevant, Time-bound) objectives:

1. **Objective 1 – Low-friction reporting:** Give students and faculty a single channel to report infrastructure issues with a photo, category, and precise structured location, deliverable as a working submission form by Week 4 of the project timeline.
2. **Objective 2 – Automatic, correctable routing:** Automatically route each report to the right department using a category-to-department mapping, while still allowing a manual override when the auto-suggested category is wrong, deliverable alongside the submission flow.
3. **Objective 3 – Duplicate prevention:** Prevent duplicate reports of the same problem from fragmenting attention by detecting likely duplicates at submission time (location, department, and description-similarity signals) and letting users upvote an existing report instead, targeted for Weeks 5–6.
4. **Objective 4 – Priority-aware escalation:** Ensure high-impact or safety-critical issues are not left to languish in a queue by automatically escalating priority once an issue's upvote count crosses a configurable threshold, targeted for Week 9.
5. **Objective 5 – Closed feedback loop:** Close the feedback loop for reporters through status-change and escalation notifications, so that "I reported it and heard nothing" is no longer the default experience.
6. **Objective 6 – Scoped administrative control:** Give department Administrators and a Super-Admin department-scoped and institution-wide visibility respectively, so ownership of an issue is always clear.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

The Civic Issue Reporter is designed as a three-tier web application (see Figure 2.1). A React single-page frontend serves two distinct experiences from the same codebase: a low-friction submission and browsing view for students/faculty, and a status-management dashboard for Administrators and the Super-Admin. All client requests pass through a stateless REST API [2] built on Node.js/Express, which is responsible for authentication and role-based access control (RBAC), issue submission and department routing, the duplicate-detection heuristic, the auto-escalation check, and notification generation. The API persists relational data – issues, users, departments, upvotes, and notifications – in a managed PostgreSQL instance [3], while photo attachments are stored separately in object storage (Cloudinary/AWS S3) so that image growth does not pressure the primary database. A CI/CD pipeline (GitHub Actions) builds, lints, and tests the backend on every change and deploys to a staging environment, in line with the project's weekly iteration cadence.

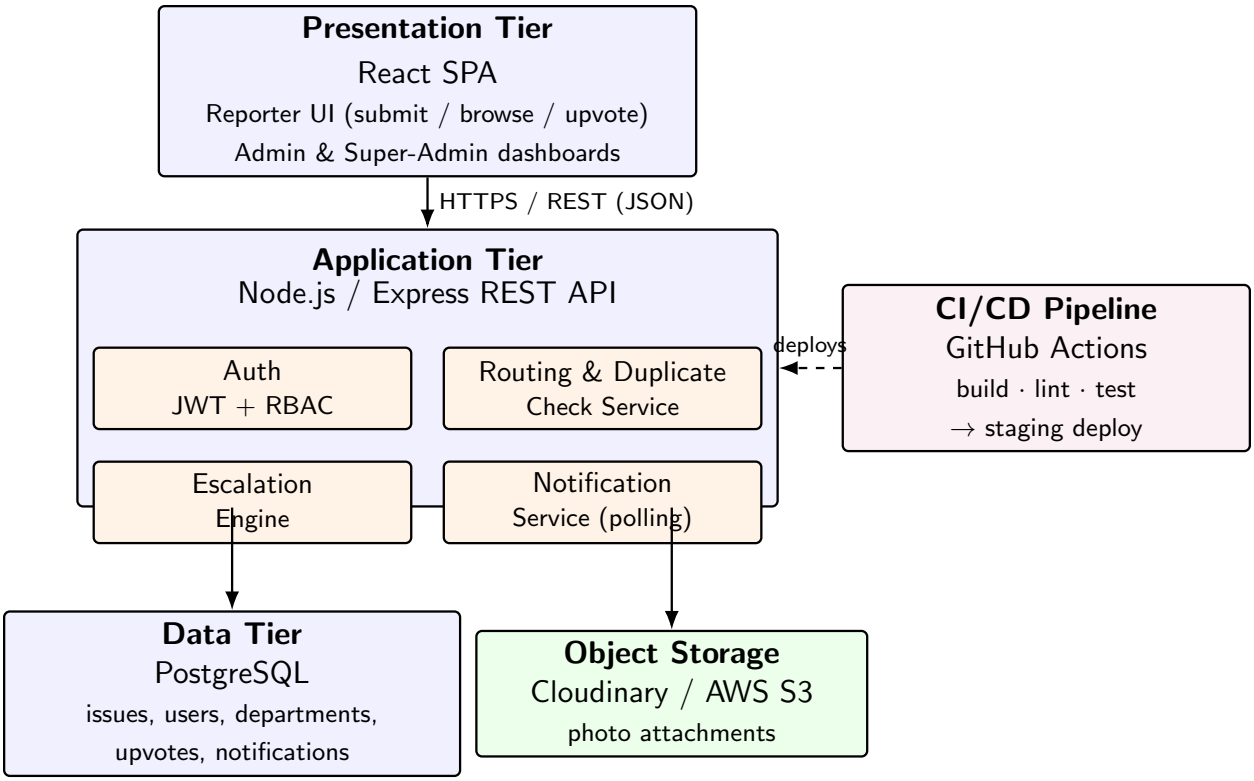


Figure 2.1: Three-tier system architecture of the Civic Issue Reporter platform.

2.1.2 Use Case View

Figure 2.2 presents the system's use-case model. Students and Faculty are modelled as functionally identical actors: both may report an issue (which includes a duplicate check), view the full list of reported issues read-only, and upvote an existing issue instead of filing a new one. An Administrator can view and update the status of issues owned by their own department, while the Super-Admin generalizes an Administrator and additionally oversees issues across *all* departments.

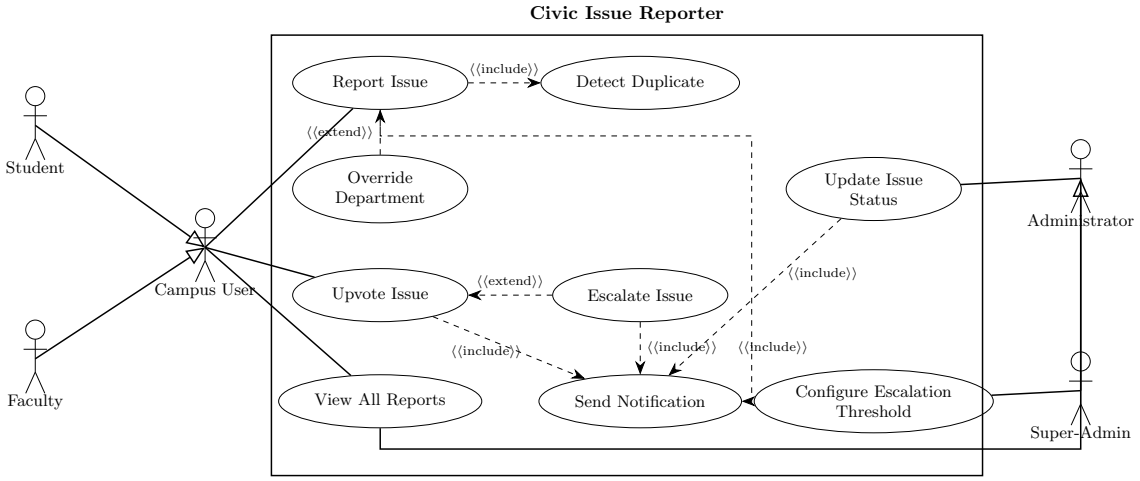


Figure 2.2: Use case diagram for the Civic Issue Reporter system.

2.1.3 Technical Stack

Tier	Technology	Purpose
Frontend	React	Responsive UI for reporters (submission form, status view) and administrators (dashboard), usable on low-to-mid range student devices.
Backend API	Node.js / Express (Flask fallback)	REST API for authentication, submission, routing, status transitions, and escalation logic.
Database	PostgreSQL	Maintains relational integrity between issues, departments, users, upvotes, and notifications; indexed on location, department, and status for duplicate detection and dashboards.
Object storage	Cloudinary / AWS S3	Stores photo attachments outside the primary database.
Auth	JWT + RBAC	Stateless session tokens with role-based access enforced at the API layer for all four roles.
CI/CD	GitHub Actions	Build, lint, and test on every change; deploy to staging.

Table 2.1: Technology choices by architectural tier.

2.2 Data Model

Figure 2.3 shows the core class/data model, drawn using standard UML class-diagram notation [4] and informed by relational data-modelling practice [5]. `User` is specialised into `Student`, `Faculty`, and `Administrator`, with `SuperAdmin` further specialising `Administrator` – reflecting that a Super-Admin is a superset of an Administrator's capabilities plus cross-department oversight. Every `Issue` belongs to exactly one `Department` and may receive many `Upvote` records, each tied to exactly one `User`; a unique constraint on the (`userId`, `issueId`) pair on `Upvote` prevents a single user from

inflating an issue's count. `Notification` records the four event types described in Section 2.3.2 and are always addressed to a single `User`.

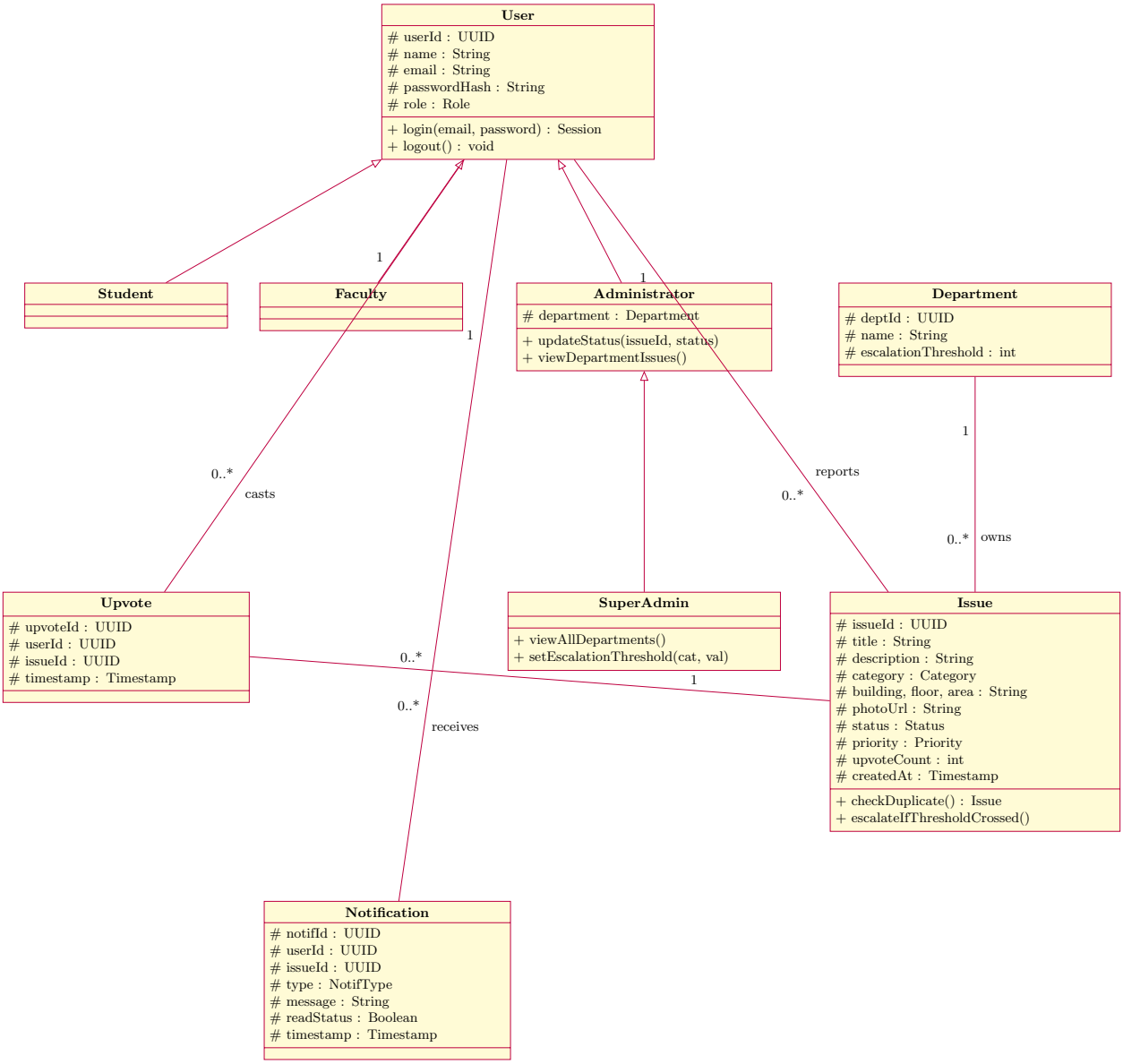
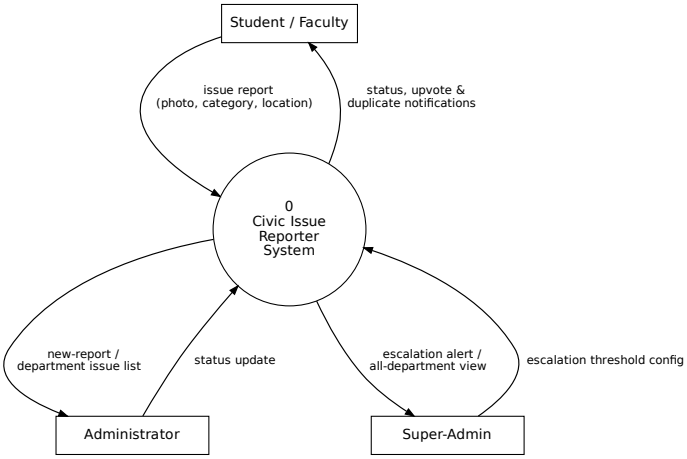


Figure 2.3: Class diagram / core data model of the Civic Issue Reporter system.

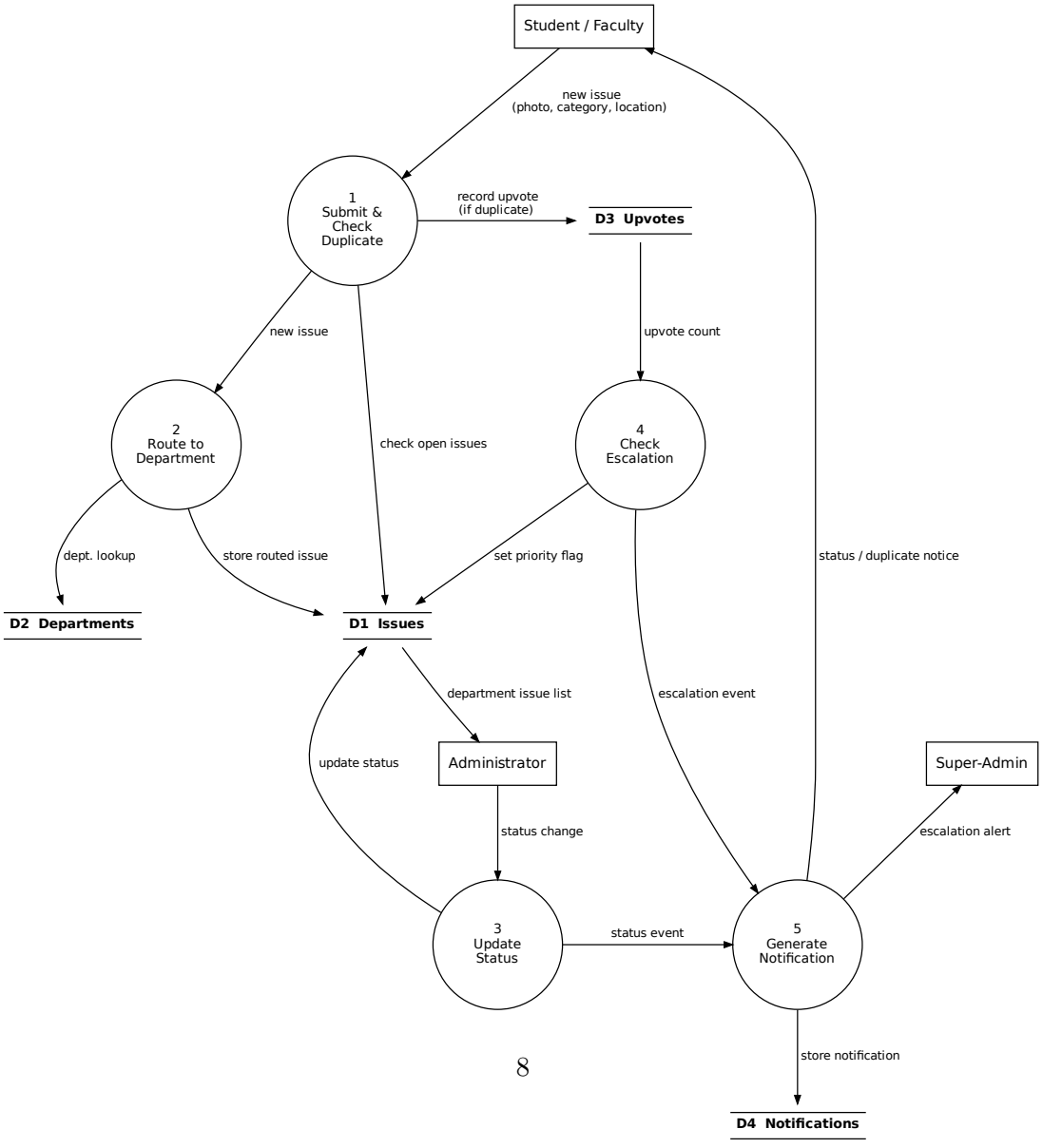
2.3 Process Model

2.3.1 Context and Level-1 Data Flow Diagrams

Figure 2.4 presents the Gane–Sarson data flow diagrams for the system. The Context Diagram (Level 0, top) fixes the system boundary and its external entities – Student/Faculty, Administrator, and Super-Admin – and the major information flows between them and the system. The Level-1 diagram (bottom) decomposes the system into its major processes (submission and duplicate check, routing, status update, and escalation/notification) and the data stores they read from or write to.



(a) Context (Level 0) data flow diagram.



2.3.2 Core Workflow

Figure 2.5 models the end-to-end reporting workflow as a UML activity diagram. A student or faculty member submits an issue with a photo, category, and structured location. The system auto-suggests a department and, in the same step, checks for likely duplicates among *open* issues in that location/department bucket. If a likely duplicate is found, the reporter is shown it and may upvote it instead of creating a new record; otherwise a new issue is created in `Reported` status and routed to the relevant Administrator, who is notified. As upvotes accumulate, the system checks the issue's count against its category's escalation threshold on every upvote event; on crossing it, the issue is marked *High Priority* and the Super-Admin is notified. The owning Administrator (or Super-Admin) subsequently moves the issue through `Reported` → `Ongoing` → `Finished`; the original reporter is notified at each transition. Four notification types are generated across this workflow: status-change (to the reporter), upvote/duplicate activity (to the reporter), new-report (to the owning department's Administrator), and escalation (to the Super-Admin).

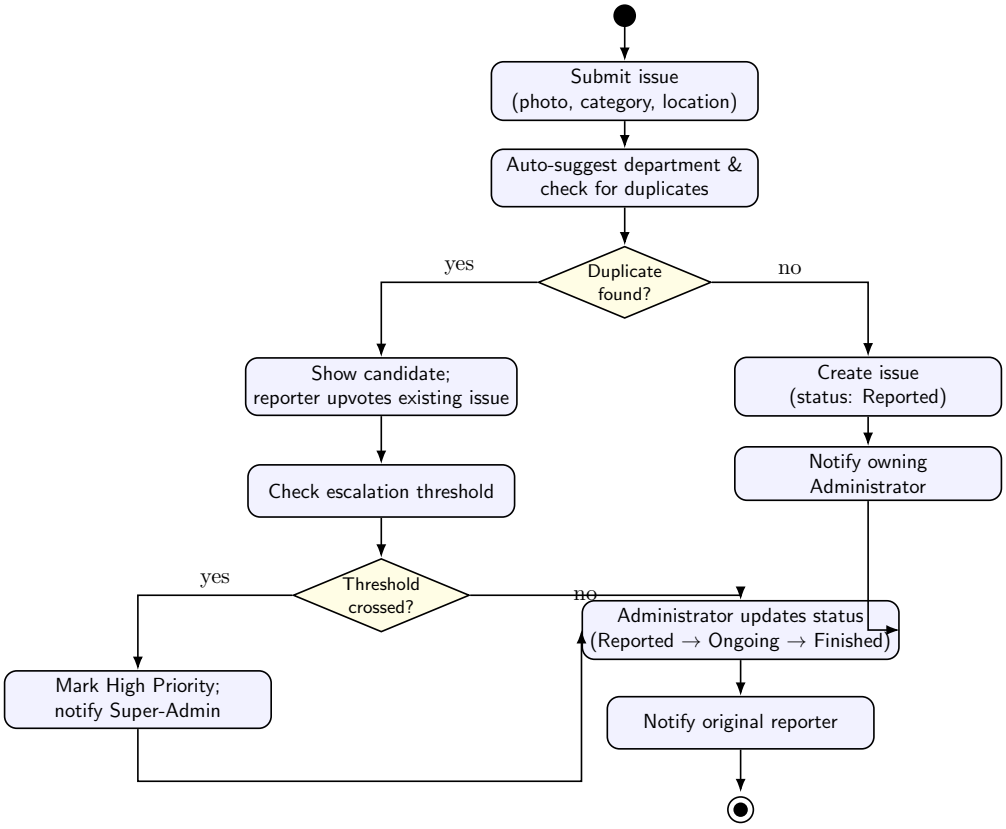


Figure 2.5: UML activity diagram for the core issue reporting and resolution workflow.

2.4 Implementation Details

2.4.1 Module 1: Reporting and Duplicate Detection

The submission module implements a structured reporting form: photo upload, category selection with an automatic department suggestion (manually overridable by the reporter or an admin), and location entry via a *Building* → *Floor* → *Area/Room* hierarchy (dropdowns plus free text), rather

than GPS or a map pin, since GPS is unreliable indoors and map APIs add cost and complexity the MVP does not need.

Duplicate detection is a heuristic combining three signals, in the spirit of duplicate bug-report detection studied in the software engineering literature [7], so that no single weak signal can produce a false match on its own:

1. **Location match** – exact match on building and floor, fuzzy match on the free-text area/room field.
2. **Department/category match** – the new report is only compared against issues already routed to the same department.
3. **Description similarity** – a token-overlap / TF-IDF cosine similarity measure, with a threshold, between the new description and existing open descriptions within the matched location/department bucket.

The comparison set is restricted to issues still open (not `Finished`), since a resolved issue recurring is a *new* problem worth a fresh report rather than a duplicate. The system surfaces the top candidate to the reporter for a human decision and never fully auto-merges, auto-deletes, or auto-rejects a submission, keeping a human in the loop for a step that directly affects data quality.

Mid-semester status. The schema-level support for this module is in place: a dedicated `duplicate_checks` table exists, along with the description, location, and department fields needed to run the three-signal match described above. The similarity-matching logic itself (NLP/TF-IDF comparison) has not been implemented yet, so at this checkpoint every submission simply creates a new issue; wiring the matching logic to this schema is scoped as post-mid-semester work (Section 2.5).

2.4.2 Module 2: Roles, Escalation, and Notifications

Access is role-based, implemented via JWT-based sessions [6] with role-based access control enforced at the API layer, matching the class hierarchy of Figure 2.3. Students and faculty share identical permissions (report, view all, upvote); Administrators are scoped to a single department and move issues through their status lifecycle; the Super-Admin has full cross-department visibility and is additionally the only role notified on every escalation event regardless of which department owns the issue. This module – roles and status transitions – is functionally complete and tested (see Table 3.1 in Chapter 3).

A single flat upvote threshold across every category would either be too high for safety-critical issues (which should not have to wait for popularity to earn attention) or too low for high-traffic areas (where "High Priority" would stop meaning anything). The design therefore makes the threshold configurable per category by the Super-Admin, with a **default of 5 upvotes** used uniformly for the MVP, since no real usage data from TIET yet exists to calibrate per-category values responsibly. This is treated as a known, temporary simplification, to be reviewed and tuned per category after the pilot.

Mid-semester status. The `priority` field on `Issue` and the per-category `escalationThreshold` field on `Department` (Figure 2.3) already exist in the database, but the application logic that checks accumulated upvote counts against the threshold and flips an issue's priority has not been written yet – this is scoped as post-mid-semester work.

Four notification types are designed to close the feedback loop described in Section 2.3.2, to be delivered via in-app polling rather than WebSockets to keep the MVP's infrastructure and failure modes simple. **Mid-semester status:** the `notifications` table and its schema are in place and can be written to, but the polling mechanism that would surface these notifications to a user in

the UI has not been built yet; this is also scoped as post-mid-semester work.

2.5 Mid-Semester Scope

To keep the mid-semester submission honest about what has actually been built versus what is merely designed, this section states explicitly which parts of Chapter 2's design are implemented end-to-end today and which are deliberately deferred to the second half of the project. The guiding principle has been *schema-first, logic-second*: the relational schema for every MVP feature was designed and migrated early (Weeks 1–2 of the timeline) so that the remaining application logic could be built incrementally against a stable data model, rather than the schema and logic being developed in lockstep feature-by-feature.

- **Duplicate detection:** the `duplicate_checks` table and the description/location/department matching structure exist in the schema, but the similarity-matching logic (NLP/TF-IDF) is not yet built. Every submission currently creates a new issue unconditionally.
- **Auto-escalation:** the `priority` field and the per-category `escalationThreshold` field exist in the database, but nothing yet automatically checks upvote counts against the threshold and flips priority – that application logic is still to be written.
- **Live notification delivery:** the `notifications` table and its write path work, but the polling mechanism that would surface these notifications to a user in the UI is not built.

These three items are precisely Objectives 3, 4, and 5 from Section 1.2, and are the team's explicit focus for the post-mid-semester phase of the timeline (Weeks 9 onward), alongside the campus pilot in Section 3.4. Everything else described in this chapter – the three-tier architecture, the role-based access model, the structured submission flow, and department routing – is implemented and already reflected in the status table in Chapter 3 (Table 3.2).

Chapter 3

Results and Evaluation

This chapter reports progress against the Chapter 2 design as of the mid-semester checkpoint (end of Week 8 in the project timeline: planning, authentication/RBAC, the submission-to-database flow, the duplicate-detection prototype, and the role-scoped dashboards). Since the campus pilot described in Section 3.4 runs later in the schedule, the metrics below distinguish between measurements already possible on the implemented modules and metrics that are defined and instrumented but whose real values will only be available once the pilot generates usage data.

3.1 Testing Methodology

- **Unit testing:** individual backend components – the department-routing function, the duplicate-similarity scorer, and the escalation-threshold check – are tested in isolation with fixed input/output cases, run automatically via the CI/CD pipeline on every commit.
- **Integration testing:** the submission-to-database flow (form → API → PostgreSQL → dashboard) and the authentication/RBAC layer are tested end-to-end to confirm that each of the four roles can perform exactly its permitted actions (Table 3.1, reproduced from the proposal).
- **Performance testing:** API response time and database query time are measured for the duplicate-check and dashboard-list endpoints, which are the two most query-heavy code paths.
- **User testing:** a small internal walkthrough with team members role-playing Student, Faculty, Administrator, and Super-Admin accounts, ahead of the bounded campus pilot.

Role	**Report**	**View all**	**Upvote**	**Change status**	**Scope**
Student	Yes	Yes	Yes	–	–
Faculty	Yes	Yes	Yes	–	–
Administrator	–	Yes	–	Yes	Own department
Super-Admin	–	Yes	–	Yes	All departments

Table 3.1: Role-based permission matrix verified by integration tests.

3.2 Implementation Status

Feature	Status	Notes
JWT auth + RBAC (4 roles)	Complete	Verified by integration tests against Table 3.1.
Structured submission form (photo, category, location)	Complete	Department auto-suggestion implemented; manual override in progress.
Duplicate-detection heuristic	Prototype	Location + department filtering complete; description-similarity threshold under tuning.
Administrator / Super-Admin dashboards	In progress	Read/list views complete; status-transition controls being wired to the API.
Auto-escalation engine	Planned	Scheduled for Week 9 per the project timeline.
Notifications (4 event types, polling)	Planned	Scheduled alongside escalation.
CI/CD pipeline (build, lint, test, staging deploy)	Complete	Runs on every push via GitHub Actions.

Table 3.2: Implementation status of MVP deliverables at the mid-semester checkpoint.

3.3 Evaluation Metrics

The following metrics are defined for the platform, each corresponding to a specific feature in Chapter 2. Values marked "pending pilot" are instrumented in the schema (e.g. status timestamps, the `read_status` field) but require real reporting volume from the campus pilot to be meaningful.

3.4 Pilot Validation Plan

Once the remaining MVP modules (escalation, notifications) are complete, the platform will be validated with a bounded pilot: one or two hostel blocks plus one academic block, rather than campus-wide, to keep reporting volume interpretable within the course timeline. A small set of student/faculty reporters will be recruited, with Administrator accounts either staffed by real departmental personnel where available or simulated by the team. An informal baseline – a short survey on how long informally-reported issues currently take to get attention – will give the metrics in Table 3.3 a comparison point, since no institutional baseline is tracked today.

3.5 Analysis

Against the six objectives in Section 1.2, Objectives 1, 2, and 6 are substantially met at this checkpoint: the reporting form, automatic routing, and role-scoped access are implemented and tested. Objective 3 (duplicate prevention) is partially met – the heuristic's structural signals (location, department) work reliably, but the description similarity threshold needs tuning against

Metric	Definition	Status
Time-to-first-action (TTFA)	Time between an issue entering `Reported` and its first transition to `Ongoing`	Pending pilot
Duplicate-detection precision	Proportion of surfaced duplicate candidates that reporters accept (upvote) rather than reject	Instrumented; preliminary internal tests show correct flagging on same-location/category test cases
Escalation precision	Proportion of auto-escalated issues an Administrator agrees warranted High Priority	Pending pilot (escalation engine not yet live)
Resolution time by category	Time from `Reported` to `Finished`, by category	Pending pilot
Duplicate deflection rate	Proportion of submissions ending as an upvote rather than a new report	Pending pilot
Notification effectiveness	Proportion of notifications marked read within a defined window	Pending pilot (notification module not yet live)

Table 3.3: Evaluation metrics and their status at the mid-semester checkpoint.

a larger and more varied set of sample descriptions than the team could generate internally, which is expected and is the reason the pilot exists. Objectives 4 and 5 (escalation and the closed notification loop) are on schedule but not yet implemented, consistent with the Week 9 target in the project timeline; no claims about their real-world behaviour can be made before they are live.

The main challenge encountered so far has been calibrating the duplicate-similarity threshold without real user-submitted description data: an overly strict threshold misses genuine duplicates, while an overly loose one produces false positives that could discourage legitimate reports. Per the risk mitigation in the original proposal, the team resolved this by always surfacing the candidate for a human decision rather than attempting to auto-tune the threshold from synthetic data, and by treating duplicate-detection precision (Table 3.3) as a live signal to revisit once pilot data exists.

Chapter 4

Conclusion

4.1 Summary of Work

This report has described the design and progressing implementation of the Civic Issue Reporter, a role-based web platform for reporting, tracking, and resolving campus infrastructure issues at TIET. Students and faculty can report problems such as electrical, plumbing, cleanliness, or other infrastructure issues with a photo, category, and structured location; department Administrators receive and manage issues routed to their department; and a Super-Admin provides institution-wide oversight. The system follows a three-tier architecture (React, Node.js/Express, PostgreSQL) with object storage for images and JWT-based role-based access control, as detailed in Chapter 2. At the mid-semester checkpoint, authentication/RBAC, the structured submission flow, a duplicate-detection prototype, and CI/CD are complete, with dashboards in progress and escalation/notifications on schedule for the following weeks (Table 3.2).

4.2 Key Findings

- **Finding 1:** Restricting duplicate comparison to *open* issues within the same location/department bucket (rather than the full historical issue table) keeps the check both semantically correct – a resolved issue recurring is a new problem – and cheap, since its cost does not grow with total historical volume.
- **Finding 2:** A single flat escalation threshold is a reasonable placeholder for an MVP but is explicitly a simplification; safety-critical and high-traffic categories genuinely need different thresholds, which is only sensible to calibrate once real upvote-volume data exists.
- **Finding 3:** Keeping a human in the loop for the duplicate decision (upvote vs. new report) turned out to be not just a safety choice but also the most practical source of a trainable precision signal, since it converts every submission into a labelled data point without requiring a separate hand-labelling effort.

4.3 Future Work and Recommendations

- Complete the auto-escalation engine and the four-type notification service (Week 9 of the timeline).
- Replace the flat MVP escalation threshold with per-category thresholds calibrated using real pilot upvote-volume data.

- Improve duplicate-detection similarity (e.g., embedding-based similarity in place of pure token overlap/TF-IDF) once enough real description data exists to evaluate it meaningfully.
- Add aggregate analytics for facilities management: recurring-issue hotspots by building/category and resolution-time trends over time.
- Consider an optional email/SMS notification channel alongside in-app notifications, and search/filtering improvements on the public issue list as report volume grows.

4.4 Final Remarks

The project's central engineering bet is that campus issue resolution is bottlenecked by *visibility and routing*, not maintenance capacity, and that a structured, trackable reporting channel can address that bottleneck without requiring any change to how maintenance work itself is carried out. The mid-semester progress supports this framing: the hardest design problems so far have been about correctly scoping the duplicate check and choosing sensible defaults under missing data, rather than about the underlying three-tier architecture, which has proceeded largely as planned. The remaining work – escalation, notifications, and the campus pilot – is what will determine whether the system's routing, duplicate handling, and prioritization actually improve outcomes in practice, and the evaluation plan in Section 3.4 has been designed specifically to make that determination measurable rather than anecdotal.

Bibliography

[1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016. https://www.pearson.com/en-us/subject-catalog/p/Sommerville-Software-Engineering-10th-Edition/P200000003258

[2] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[3] The PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2024. https://www.postgresql.org/docs/

[4] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Addison-Wesley, 2004. https://www.oreilly.com/library/view/uml-distilled-a/0321193687/

[5] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2015. https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003546

[6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519.html

[7] N. Bettenburg, R. Premraj, T. Zimmermann, and S. Kim, "Duplicate Bug Reports Considered Harmful ... Really?," in *Proc. 24th IEEE Int. Conf. Software Maintenance (ICSM)*, Beijing, China, 2008, pp. 337–345, doi:10.1109/ICSM.2008.4658082. https://thomas-zimmermann.com/publications/files/bettenburg-icsm-2008.pdf